

API — GateFlow

Versión 1.0 · Depende de: 02-ARCHITECTURE.md, 03-DATABASE.md, 01-BUSINESS_RULES.md · Precede a: 05-UX.md (para contratos que la UI consumirá), 18-API_GUIDELINES.md (que detallará convenciones de estilo)

Este documento define el contrato de la API a nivel conceptual: qué existe, cómo se organiza, cómo se autentica y cómo se comporta. No contiene implementación ni ejemplos de código ejecutable.

1. Principios de diseño

- **La API no es una capa que se escribe desde cero — es en gran parte generada.** Al usar Supabase, la mayoría de las operaciones CRUD sobre el núcleo de plataforma y sobre el módulo Paquetes quedan expuestas automáticamente vía PostgREST, respetando las políticas RLS ya definidas. Escribir un controlador manual para “listar paquetes” sería duplicar trabajo que el motor ya resuelve correctamente.
- **Las Edge Functions existen solo donde hay lógica que no puede vivir en una simple consulta a una tabla.** Generación de código GateFlow, envío de notificaciones WhatsApp, recálculo de agregaciones del dashboard, y en el futuro, invocación del servicio de OCR — estas son las candidatas legítimas a función propia; todo lo demás se resuelve con el API autogenerado.
- **La autoridad de seguridad es la base de datos, no la API.** Ningún endpoint debe re-implementar verificación de tenant o de rol en código de aplicación; esa verificación ya ocurre en RLS. La API confía en la base de datos, no al revés.
- **Un contrato estable no significa un contrato congelado.** Se versiona explícitamente para poder evolucionar sin romper clientes existentes (guardias con app desactualizada en campo, por ejemplo).

2. Organización de recursos

Los recursos se organizan siguiendo la misma separación núcleo/módulo definida en 03-DATABASE.md :

Recursos núcleo de plataforma: tenants, usuarios, roles, permisos, unidades, calles, manzanas, edificios, residentes.

Recursos del módulo Paquetes: paquetes, fotografías de paquete, firmas, incidencias, ubicaciones, catálogos de tamaño/prioridad, empresas de paquetería, notificaciones.

Recursos transversales (no pertenecen a un módulo, sirven a todos): búsqueda universal, dashboard/métricas, auditoría.

Cada módulo futuro (Visitantes, Accesos, etc.) añadirá su propio conjunto de recursos siguiendo este mismo patrón, sin modificar los recursos existentes.

3. Autenticación y autorización

- Toda solicitud autenticada porta un token JWT emitido por Supabase Auth, con `tenant_id` y `role` incluidos como claims (vía Auth Hook), consistente con lo definido en `02-ARCHITECTURE.md`.
- No existe un endpoint de la API que reciba `tenant_id` como parámetro de la solicitud para decidir qué datos devolver — el tenant siempre se deriva del token, nunca del request. Esto elimina por diseño la posibilidad de que un cliente mal configurado o comprometido solicite datos de un tenant ajeno simplemente cambiando un parámetro.
- Las operaciones que requieren privilegios de Super Admin GateFlow (gestión de tenants, soporte) se exponen en un espacio de rutas administrativo separado, con su propia verificación de rol, nunca mezcladas con las rutas de un tenant normal.

4. Versionado

- Las Edge Functions (lógica custom) se versionan explícitamente en su ruta. Un cambio incompatible se publica como una nueva versión, y la versión anterior se mantiene activa durante un período de transición para no romper dispositivos de guardias que no han actualizado su app.
- El API autogenerado por PostgREST sobre las tablas no requiere versionado propio en esta etapa: los cambios de esquema se gestionan como adiciones no destructivas (nuevas columnas opcionales, nunca eliminación o renombrado de campos consumidos por clientes activos), consistente con la filosofía de no-destrucción de datos ya establecida en las reglas de negocio.

5. Formato de respuesta

Toda respuesta, exitosa o no, sigue una envoltura consistente con tres partes conceptuales: los datos solicitados (cuando aplica), un bloque de error (nulo cuando la operación fue exitosa) y metadatos de la respuesta (paginación, timestamp del servidor). Esto evita que

cada endpoint invente su propia forma de reportar éxito o fallo, y permite que la app de guardia y el panel de administración compartan la misma lógica de manejo de respuestas.

6. Manejo de errores

Los errores se clasifican en categorías consistentes, independientemente del recurso involucrado:

Categoría	Cuándo ocurre	Comportamiento esperado del cliente
Error de validación	El registro no cumple una regla de negocio (ej. intentar entregar un paquete ya entregado, BR-14).	Mostrar el mensaje específico al usuario; no reintentar automáticamente.
Error de autorización	El usuario no tiene permiso para la acción solicitada.	No reintentar; no debe ocurrir en la operación normal de la UI si los permisos se reflejan correctamente en la interfaz.
Error de conectividad	El dispositivo no puede alcanzar el servidor.	La operación queda en cola local (ver sección 8) y se reintentará automáticamente al recuperar conexión — nunca se le pide al guardia que reintente manualmente.
Error del servidor	Falla interna no relacionada con la solicitud del cliente.	Reintento automático con backoff; si persiste, se registra para revisión operativa.

7. Endpoints del núcleo de plataforma (indicativo, no exhaustivo)

Recurso	Operaciones	Quién puede invocarlas
Tenants	Consulta del tenant propio; creación reservada a onboarding self-service (V1.0).	Admin de Residencial (lectura de su propio tenant); Super Admin (gestión completa).
Usuarios y roles	Alta, edición y consulta de usuarios dentro del tenant.	Admin de Residencial.

Unidades (casas/departamentos)	Alta individual, importación masiva (V1.0), consulta.	Admin de Residencial.
Residentes por unidad	Alta, edición de vigencia, consulta.	Admin de Residencial.

8. Endpoints del módulo Paquetes (indicativo, no exhaustivo)

Recurso	Operaciones	Quién puede invocarlas
Paquetes	Registro de recepción, consulta, cambio de estado, registro de entrega.	Guardia (registro y entrega); Admin (consulta y reportes).
Fotografías de paquete	Subida asociada a un paquete existente.	Guardia.
Firmas	Registro asociado a una entrega.	Guardia.
Incidencias	Alta, consulta, cambio de estado (abierta/en seguimiento/resuelta).	Guardia (alta); Admin (gestión y cierre).
Ubicaciones	Configuración de la jerarquía de almacenamiento del tenant.	Admin de Residencial.
Catálogos (tamaño, prioridad, empresas de paquetería)	Configuración de valores propios del tenant.	Admin de Residencial.
Generación de código GateFlow	Solicitud de bloque de códigos por dispositivo (Edge Function).	Guardia, automáticamente al iniciar turno con conectividad.
Notificaciones	Disparo automático al cambiar estado a "recibido"; consulta de historial de envíos.	Sistema (disparo automático); Admin (consulta).

9. Recursos transversales

Recurso	Operaciones	Quién puede invocarlas
Búsqueda	Consulta de texto libre sobre la vista de	Guardia y Admin, siempre acotado

universal	búsqueda unificada definida en 03-DATABASE.md .	a su propio tenant.
Dashboard / métricas	Consulta de la capa de agregación pre-calculada.	Admin de Residencial.
Auditoría	Consulta de solo lectura del historial de cambios.	Admin de Residencial (su tenant); Super Admin (con su propio acceso auditado).

10. Paginación y filtrado

Todo endpoint que devuelve colecciones (paquetes, incidencias, usuarios, resultados de búsqueda) implementa paginación consistente y filtrado por los campos relevantes de cada recurso (ej. paquetes por estado, por rango de fecha, por unidad). El límite de resultados por página tiene un valor por defecto razonable para el uso en campo (listados cortos, de carga rápida en el dispositivo del guardia) y uno distinto, mayor, para el uso administrativo (reportes, exportaciones).

11. Relación con la capa de sincronización offline

La mayoría de las operaciones de escritura del guardia (registrar paquete, subir foto, capturar firma) **no pasan por esta API REST convencional en el momento de la captura** — pasan por la capa de sincronización offline descrita en 02-ARCHITECTURE.md , que replica los cambios hacia Postgres respetando las mismas políticas RLS. La API REST tradicional sigue siendo el punto de entrada para: el panel de administración (que asume conexión), las Edge Functions de lógica custom, y las consultas de solo lectura que la app de guardia haga cuando sí tiene conectividad disponible (ej. refrescar el listado de “recientes”). Este documento describe el contrato de esa capa REST/Edge Functions; el comportamiento del motor de sincronización en sí se rige por lo definido en 02-ARCHITECTURE.md .

12. Rate limiting

Se aplican límites de tasa en los endpoints de autenticación y en las Edge Functions de mayor costo (envío de notificaciones, generación de bloques de código), para proteger el sistema de un tenant con comportamiento anómalo sin afectar a otros tenants — consistente con el aislamiento multi-tenant ya establecido. Los límites concretos (umbral numérico, ventana de tiempo) se definen en la fase de implementación, no en este documento conceptual.

13. Documentación viva del contrato

El contrato autogenerado por PostgREST se documenta automáticamente a partir del esquema de base de datos (OpenAPI generado por Supabase); las Edge Functions custom requieren documentación manual explícita de su firma de entrada/salida, mantenida junto al código de cada función. Este documento (`04-API.md`) describe la intención y organización general; no sustituye la referencia autogenerada ni la documentación específica de cada función, que vivirán junto al código cuando este se escriba.

14. Extensión para módulos futuros

Cada módulo nuevo añade su propio bloque de endpoints siguiendo la tabla de la sección 8 como plantilla (recurso, operaciones, quién puede invocarlas), sin alterar los bloques de secciones 7 y 9. El recurso de búsqueda universal (sección 9) deberá extenderse para incluir los campos relevantes de cada módulo nuevo, consistente con la regla de negocio de mantener un único buscador (BR-42) en lugar de uno por módulo.